

# Autonomous Software Agents: Deliveroo.js

Davide Donà – `davide.dona-1@studenti.unitn.it`  
264467

Andrea Blushi – `andrea.blushi@studenti.unitn.it`  
264468

University of Trento

June 2026



# Contents

- 1 Introduction
- 2 BDI Agent
- 3 PDDL Crate Navigation
- 4 LLM Agent Layer

# Our Solution

- **BDI core**: sense  $\rightarrow$  deliberate  $\rightarrow$  plan  $\rightarrow$  act; collects, explores, navigates
- **PDDL planner**: pushes crates when A\* is blocked
- **LLM layer**: interprets mission commands; injects goals, rules, penalties
- **Coordination**: 2PC protocols, peer-injection, team coordinator

# Beliefs: Tracker

- Stores latest observation  $\langle v, t_{seen} \rangle$  per entity
- Updated on each sensing report
- Invalidated when entity is visible-but-absent

## Confidence decay

$$c(t) = \exp\left(-\frac{t - t_{seen}}{\tau}\right)$$

# Beliefs: Memory

- Bounded temporal history of observations per entity
- Entries older than a configurable TTL are evicted automatically
- Tracks enemy position history for heat-field computation

# Beliefs: Agents

- **Self**: own position and current state
- **Friends**: latest state per teammate via Tracker
- **Enemies**: latest state via Tracker; position history via Memory
- **EnemyPredictor**: estimates each enemy's next position from history
- **Spatial heat field**: aggregates past enemy observations across the map

# Beliefs: Map & Crates

- Static tile layout, transient blocked tiles, traversal penalties
- Crates tracked as dynamic obstacles; latest position via Tracker;

# Beliefs: Map - Safe Zones

Directional Tiles create one-way edges, may trap agents in dead zones

## Safe SCC (Tarjan's algorithm)

- Map is represented as a directed graph with edges for valid movements;
- Tarjan's algorithm identifies SCCs in this graph;
- An SCC is *safe* iff it contains  $\geq 1$  spawn tile *and*  $\geq 1$  delivery tile;
- All tiles outside safe SCCs are reclassified as walls.

*Reminder: an SCC (strongly connected component) is a maximal set of nodes where every node is reachable from every other along directed edges.*



# Beliefs: Map - Safe Zones

*[Safe SCC demo]*

# Beliefs: Parcels

- Reward decay is applied even when out of sensor range
- Parcel  $p$  removed from beliefs when  $r_p(t) \leq 0$

## Estimated reward

$$r_p(t) = r_{p,0} - \left\lfloor \frac{t - t_0}{\Delta_t} \right\rfloor$$

# Desire: Distance Metric

- Manhattan distance poorly correlates with actual path;
- BFS from target tile computes true shortest path distance accounting for walls and crates;
- This gives a more accurate heuristic for desire scoring.

# Desire: ReachParcel

- Navigate to and collect a visible parcel
- Priority 1: competes directly with DeliverParcel

## Score

$$S = c_p \cdot \frac{r_p + \sum_{c \in C} r_c}{d} \cdot \gamma, \quad \gamma = \min\left(1, \frac{d_{enemy}}{d}\right)$$

- $r_p$ : parcel reward;
- $\sum_{c \in C} r_c$ : value of carried stack;
- $c_p$ : confidence that parcel is still at its last known position;
- $\gamma$ : race factor that penalizes parcels where enemies have advantage;

## Desire: DeliverParcel

- Navigate to delivery tile while carrying parcels
- Priority 1: competes directly with ReachParcel

Score

$$S = \frac{\sum_{c \in C} r_c}{d}$$

# Desire: Explore

- Seek unvisited spawn tile when no parcels are reachable
- Priority 0: fallback;
- $w$ : cluster density;  $\tau$ : freshness;  $h$ : enemy heat

Score

$$S = \frac{w \cdot \tau}{d \cdot (1 + h)}$$

## Desire: Explore - Variables

Freshness  $\tau$

$$\tau = \min\left(1, \frac{\Delta t}{T_{\text{spawn}}}\right)$$

$\Delta t$ : time since spawn tile last observed;  $T_{\text{spawn}}$ : spawn interval

Cluster weight  $w$

$$w = \sum_{y \in OST} \frac{1}{\delta(x, y) + 1}$$

$OST$ : spawn tiles within observation range;  $\delta(x, y)$ : BFS distance from  $x$  to  $y$

Enemy heat  $h$

$$h = \sum_{e \in E} \exp\left(-\frac{d_e^2}{2\sigma^2}\right) \exp\left(-\frac{\Delta t_e}{\tau_h}\right)$$

Spatial  $\times$  temporal decay over tracked enemy observations

# Collision: Detection

Checked before every move step. Target tile is *occupied* if:

- An enemy is currently observed on it, *or*
- An enemy's *predicted* next position matches it with sufficient confidence

## Enemy position prediction

Next position estimated from last known state and movement history; only predictions above a confidence threshold trigger a block.



# Collision: Escalation

- ① **Detour:** replan with tile as wall;  
accepted only if extra steps  $\leq \Delta_{th}$
- ② **Wait:** no viable detour:  
pause for random duration  $\sim \mathcal{U}$
- ③ **Block & replan:** after wait expires  
or after  $L$  repeated invalidations:  
mark tile blocked for random TTL, full  
replan

*[collision resolution]*

# Collision with Friends

- The escalation above applies only to enemies;
- Asymmetric by agent id: the *lower-id* agent yields;

# When A\* Fails

- A\* fails: no path that avoids crates
- Second A\* ignoring crates finds a “ghost” path
- Crates intersecting that path are part of the “PDDL” problem instance

# PDDL

- Cluster detected → PDDL problem built from current beliefs
- Solver finds move + push sequence to reach goal
- Result translated back to agent's action plan

*[PDDL demo]*

# Performance Bottlenecks

- **Real-Time Constraints:** The initial PDDL solver integration was too slow to meet the game's strict timing demands.
- **Identified Issues & Fixes:**
  - **API Overhead:** Remote solver calls introduced heavy latency. **Fix:** Migrated to a local solver instance.
  - **Redundant Computation:** Identical crate configurations were being solved repeatedly, but implementing a standard cache was non-trivial.
- **The Pivot:** Rather than brute-forcing a complex caching mechanism, we restated the problem to naturally maximize cache hits.

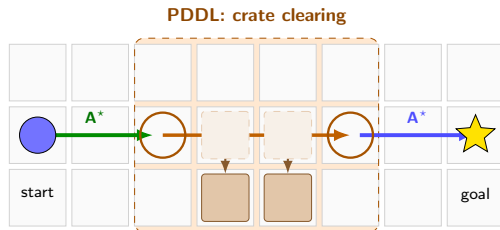
# Optimization: Problem Restatement

## Three-Phase Journey Segmentation

- ① **Approach:** A\* pathfinding from the current position to the cluster *entry*.
- ② **Manipulation:** PDDL execution solely through the crate cluster (entry → exit).
- ③ **Completion:** A\* pathfinding from the cluster *exit* to the final goal.

## Optimization: Problem Restatement

- **The Result:** The PDDL problem is now strictly anchored to the cluster entry tile rather than absolute map coordinates.
- **Simplified Cache Key:**  $\langle \text{entry, exit, \{crate positions\}} \rangle$ .



# LLM Layer Overview

- Thin wrapper around the BDI core
- Listens and interprets mission-controller chat messages
- Modifies the BDI behaviour accordingly
- Four command categories: **goal injection**, **rules**, **coordination** and **cooperation**

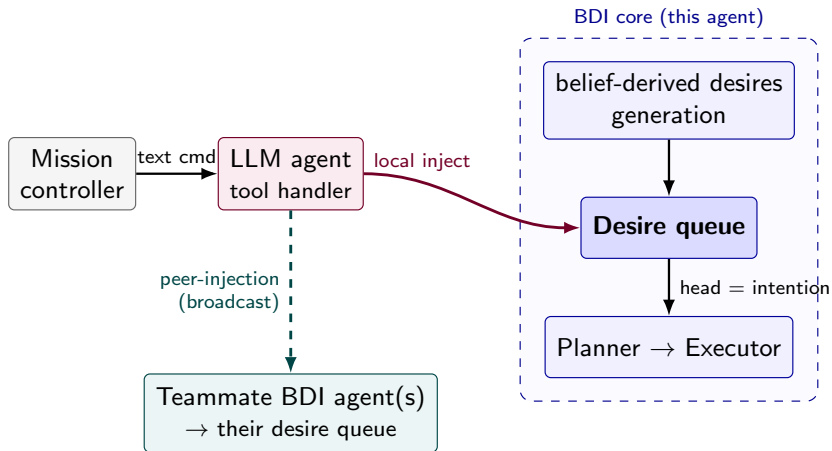


# Goal Injection

## Trigger Examples

- *“Move to coordinate (4,7) and you get +10pts”*
  - *“Drop a package in the leftmost tile to get 5pt”*
- 
- LLM can inject new goals into the BDI agent (e.g. `go-to`, `deliver-at`)
  - Compete with existing BDI desires; may be accepted or rejected
  - Persist and re-evaluated until executed or TTL expires
  - **Peer-injection:** the LLM applies the command locally, then repackages it and broadcasts it

# Goal Injection: Local + Peer Broadcast



# Rule Injection

## Trigger Examples

- *"Deliver stacks of exactly 3 parcels at a time to double the reward"*
  - *"Every time you deliver in  $(x1,y1)$  you get 0 pts"*
- 
- **RuleStore**: Defines a condition, multiplier ( $m$ ), and offset ( $a$ )
  - **Effect**: Dynamically scales and shifts the BDI score for any desire matching the condition
  - **Special case**: The **Stack rule** automatically elevates **Explore** to priority 1

# Coordination: Position Sharing

- **Beacons:** each agent broadcasts position at fixed interval
- Ensures up-to-date peer positions beyond sensing radius
- Needed for coordination when agents are outside each other's sensing radius

# Coordination: 2-Phase Commit

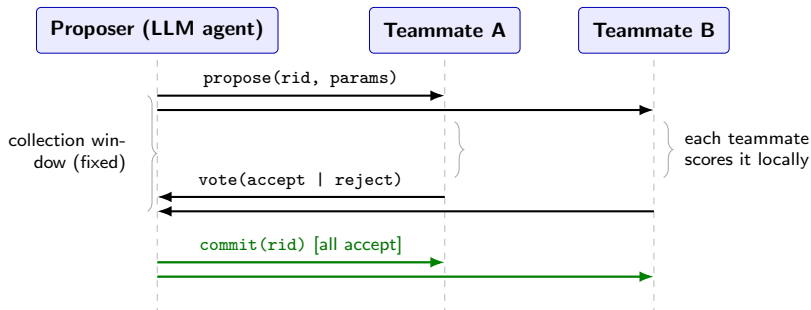
## Trigger Example

- *“Move both agents to the neighborhood of position  $(x,y)$  within a maximum distance of 3, and have them wait for each other. You will receive 500pts.”*

Used for complex coordination tasks (commit together or not at all):

- **Phase 1 - Propose:** proposer broadcasts candidate hold tiles; each agent scores them
- **Vote:** *accept* if the hold outweighs current desires; *reject* if current desires are better
- **Phase 2 - Decide:** unanimous accept  $\rightarrow$  commit (all inject); any miss or reject  $\rightarrow$  abort

# Coordination: 2-Phase Commit



# Cooperation: Motivation

## Origin: Admin Directive

*"If a parcel is initially picked up by one agent and later delivered by the other agent, you will receive a 200 points bonus."*

- A one-shot *check-and-coordinate* is insufficient: requires dynamic pickups, meeting tile computations, and joint commitments.
- ⇒ Solution: Dedicated **roles** with specific behaviours, agreed upon via a **2-phase commit** (2PC).
- Generalised into a reusable framework of **strategies** and **roles**.

# Cooperation: Generalisation

- A **cooperation round** opens every  $\Delta t$  seconds (or immediately upon a mission directive).
- Agents request teammates' beliefs to build a **shared compact context**:

## Context State

- **Roster**: Positions, payloads, sensed enemies.
  - **Map Geometry**: Spawn/delivery clusters, separation, corridor flags.
  - **Performance**: Average score/round per strategy (sliding window).
- ⇒ The **LLM** evaluates this context to select the optimal shared strategy for the team.



# Cooperation: Strategies

- ZONAL\_RELAY: One agent sweeps the spawn region and drops at a fixed midpoint; the companion ferries parcels to delivery.
- OPPORTUNISTIC: Agents forage independently and hand off parcels dynamically upon pickup.
- NONE: Fully autonomous baseline (no cooperation).

# Cooperation: Roles

- Each strategy assigns every agent a specific **role**.
- Standard BDI desire generation is *blocked*.
- A role-specific **FSM** generates desires dynamically based on the shared context and internal state.
- Meeting points and positioning are computed *deterministically* from the map's geometry.

# Cooperation: Handpass

Both strategies culminate in a **handpass**:

- ZONAL\_RELAY:  
The exchange loops indefinitely at the designated midpoint.
- OPPORTUNISTIC:  
The agent acquiring a parcel proposes a handpass via 2PC.

*[Collaboration demo]*

**Thank you for your attention!**